

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА» (БГТУ им. В.Г. Шухова)
Кафедра программного обеспечения
вычислительной техники и автоматизированных
систем

Лабораторная работа №8
по дисциплине: БД
тема: «Оптимизация sql-запросов в СУБД с использованием
планировщика»

Выполнил: студент группы ВТ-231
Масленников Д. А.
Проверили:
Панченко М. В.

Белгород 2025

Цель работы: получение навыков повышения производительности работы СУБД с помощью оптимизации sql-запросов с использованием планировщика.

Вариант 14

Задание к работе

1. Сгенерировать данные для таблиц, используемых в sql-запросах лабораторной работы №4 (1000 записей).
2. Построить планы исполнения запросов, созданных в процессе выполнения лабораторной работы №4.
3. Повлиять на производительность с использованием методов управления планировщиком. Предложить способы ускорения выполнения запросов. Продемонстрировать планы исполнения запросов.
4. Провести сравнительный анализ выполнения запросов при 100 / 1000 записей в таблицах.

Структура моей базы данных из предыдущих работ

```
railway=# \dt
```

Список отношений			
Схема	Имя	Тип	Владелец
public	car	таблица	postgres
public	contactdetails	таблица	postgres
public	passenger	таблица	postgres
public	route	таблица	postgres
public	schedule	таблица	postgres
public	seat	таблица	postgres
public	ticket	таблица	postgres
public	ticketstatus	таблица	postgres
public	train	таблица	postgres

(9 строк)

Скрипт для генерации данных:

```
import psycopg2
import random
from datetime import datetime, timedelta
from faker import Faker

fake = Faker('ru_RU')

# Connect to the database
conn = psycopg2.connect(
    dbname='railway',
    user='postgres',
    password="", # Adjust if needed
    host='localhost'
)
cur = conn.cursor()

# Helper functions (same as before)

def random_string(length=10):
    letters = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'
    return ''.join(random.choice(letters) for _ in range(length))

def random_phone():
    return '+7' + ''.join(str(random.randint(0, 9)) for _ in range(10))

def random_email(i):
```

```

return f'user{i}@example.com'

def random_passport():
    return f'{random.randint(1000, 9999)} {random.randint(100000, 999999)}'

def random_datetime():
    start = datetime(2025, 1, 1)
    end = datetime(2025, 12, 31)
    delta = end - start
    random_days = random.randint(0, delta.days)
    random_hours = random.randint(0, 23)
    random_minutes = random.randint(0, 59)
    return start + timedelta(days=random_days, hours=random_hours, minutes=random_minutes)

# Insert TicketStatus
statuses = ['активен', 'возвращен', 'забронирован']
for status in statuses:
    cur.execute("INSERT INTO TicketStatus (StatusName) VALUES (%s) ON CONFLICT (StatusName) DO NOTHING", (status,))
conn.commit()

cur.execute("SELECT ID FROM TicketStatus ORDER BY StatusName")
status_ids = [row[0] for row in cur.fetchall()]

# Insert Train (100 records)
train_types = ['скорый', 'пассажирский', 'фирменный']
train_ids = []
for i in range(100):
    train_number = f"{i+1:03}{random.choice('ABCDEFGF')}"
    train_type = random.choice(train_types)
    cur.execute("INSERT INTO Train (TrainNumber, TrainType) VALUES (%s, %s) RETURNING ID", (train_number, train_type))
    train_ids.append(cur.fetchone()[0])
conn.commit()

# Insert Route (200 records)
stations = ['Москва', 'Санкт-Петербург', 'Казань', 'Екатеринбург', 'Новосибирск', 'Владивосток', 'Сочи', 'Калининград', 'Мурманск', 'Волгоград'] + [fake.city() for _ in range(20)] # More stations
for variety
route_ids = []
inserted_routes = set()
for _ in range(500): # To get ~200 unique
    dep = random.choice(stations)
    arr = random.choice([s for s in stations if s != dep])
    if (dep, arr) in inserted_routes:
        continue

```

```

        inserted_routes.add((dep, arr))
        distance = random.randint(100, 5000)
        duration = random.randint(60, 1440)
        cur.execute("INSERT INTO Route (DepartureStation, ArrivalStation, Distance,
DurationMinutes) VALUES (%s, %s, %s, %s) RETURNING ID", (dep, arr, distance, duration))
        route_ids.append(cur.fetchone()[0])
        if len(route_ids) >= 200:
            break
    conn.commit()

```

```

# Insert ContactDetails (100000 records)
contact_ids = []
batch_size = 1000
for batch in range(0, 100000, batch_size):
    values = []
    for i in range(batch + 1, batch + batch_size + 1):
        fullname = f'Контакт {random_string(5)} {random_string(6)}'
        phone = random_phone()
        email = random_email(i)
        passport = random_passport()
        values.append((fullname, phone, email, passport))
    cur.executemany("INSERT INTO ContactDetails (FullName, Phone, Email, PassportData)
VALUES (%s, %s, %s, %s) RETURNING ID", values)
    contact_ids.extend([row[0] for row in cur.fetchall()])
conn.commit()

```

```

# Insert Passenger (100000 records)
passenger_ids = []
for contact_id in contact_ids:
    cur.execute("INSERT INTO Passenger (ContactDetailsID) VALUES (%s) RETURNING ID",
(contact_id,))
    passenger_ids.append(cur.fetchone()[0])
conn.commit()

```

```

# Insert Schedule (10000 records)
schedule_ids = []
for _ in range(10000):
    train_id = random.choice(train_ids)
    route_id = random.choice(route_ids)
    dep_time = random_datetime()
    duration_min = random.randint(60, 1440)
    arr_time = dep_time + timedelta(minutes=duration_min)
    cur.execute("INSERT INTO Schedule (TrainID, RouteID, DepartureDateTime,
ArrivalDateTime) VALUES (%s, %s, %s, %s) RETURNING ID", (train_id, route_id, dep_time,
arr_time))
    schedule_ids.append(cur.fetchone()[0])

```

```

conn.commit()

# Insert Car (50000 records)
car_types = ['плацкарт', 'купе', 'СВ', 'сидячий']
car_ids = []
car_seats = {}
for _ in range(50000):
    schedule_id = random.choice(schedule_ids)
    car_number = random.randint(1, 20)
    car_type = random.choice(car_types)
    total_seats = random.randint(18, 81)
    cur.execute("INSERT INTO Car (ScheduleID, CarNumber, CarType, TotalSeats) VALUES (%s, %s, %s, %s) RETURNING ID", (schedule_id, car_number, car_type, total_seats))
    car_id = cur.fetchone()[0]
    car_ids.append(car_id)
    car_seats[car_id] = total_seats
conn.commit()

# Insert Seat (500000 records)
seat_types = ['сидячее', 'лежащее']
seat_ids = []
for car_id, total_seats in car_seats.items():
    for seat_num in range(1, total_seats + 1):
        seat_type = random.choice(seat_types)
        cur.execute("INSERT INTO Seat (CarID, SeatNumber, SeatType) VALUES (%s, %s, %s) RETURNING ID", (car_id, seat_num, seat_type))
        seat_ids.append(cur.fetchone()[0])
conn.commit()

# Insert Ticket (100000 records)
for batch in range(0, 100000, batch_size):
    values = []
    for _ in range(batch_size):
        passenger_id = random.choice(passenger_ids)
        seat_id = random.choice(seat_ids)
        status_id = random.choice(status_ids)
        sale_time = random_datetime()
        price = round(random.uniform(1000, 10000), 2)
        values.append((passenger_id, seat_id, status_id, sale_time, price))
    cur.executemany("INSERT INTO Ticket (PassengerID, SeatID, StatusID, SaleDateTime, Price) VALUES (%s, %s, %s, %s, %s)", values)
conn.commit()

cur.close()
conn.close()

```

```
print("Данные сгенерированы успешно (100k+).")
```

Первый запрос:

```
EXPLAIN ANALYZE
SELECT
  CASE
    WHEN EXTRACT(MONTH FROM s.DepartureDateTime) IN (12, 1, 2) THEN 'Зима'
    WHEN EXTRACT(MONTH FROM s.DepartureDateTime) IN (3, 4, 5) THEN 'Весна'
    WHEN EXTRACT(MONTH FROM s.DepartureDateTime) IN (6, 7, 8) THEN 'Лето'
    WHEN EXTRACT(MONTH FROM s.DepartureDateTime) IN (9, 10, 11) THEN 'Осень'
  END AS season,
  r.DepartureStation,
  r.ArrivalStation,
  COUNT(t.ID) AS passengers_count
FROM Ticket t
JOIN Seat st ON t.SeatID = st.ID
JOIN Car c ON st.CarID = c.ID
JOIN Schedule s ON c.ScheduleID = s.ID
JOIN Route r ON s.RouteID = r.ID
JOIN TicketStatus ts ON t.StatusID = ts.ID
GROUP BY season, r.DepartureStation, r.ArrivalStation
ORDER BY season, passengers_count DESC;
```

В результате получаем:

```
Planning Time: 2.515 ms
Execution Time: 1.251 ms
```

После добавляем индексы для данных запросов:

```
CREATE INDEX IF NOT EXISTS idx_schedule_trainid ON Schedule(TrainID);
CREATE INDEX IF NOT EXISTS idx_schedule_routeid ON Schedule(RouteID);
CREATE INDEX IF NOT EXISTS idx_car_scheduleid ON Car(ScheduleID);
CREATE INDEX IF NOT EXISTS idx_seat_carid ON Seat(CarID);
CREATE INDEX IF NOT EXISTS idx_ticket_seatid ON Ticket(SeatID);
```

И повторяем наш запрос, получая результат

```
Planning Time: 1.132 ms
Execution Time: 0.785 ms
```

Такое же сравнение для второго запроса:

```
EXPLAIN ANALYZE
SELECT
  tr.TrainNumber,
  tr.TrainType,
  c.CarType,
  COUNT(t.ID) AS tickets_sold
FROM Ticket t
JOIN Seat st ON t.SeatID = st.ID
JOIN Car c ON st.CarID = c.ID
JOIN Schedule s ON c.ScheduleID = s.ID
JOIN Train tr ON s.TrainID = tr.ID
JOIN TicketStatus ts ON t.StatusID = ts.ID
GROUP BY tr.TrainNumber, tr.TrainType, c.CarType
ORDER BY c.CarType, tickets_sold DESC;
```

До добавления индексов

Planning Time: 1.964 ms
Execution Time: 4.049 ms

После:

Planning Time: 1.310 ms
Execution Time: 3.036 ms

Вывод: в ходе выполнения лабораторной работы мы на практическом опыте познакомились с индексами. Так же получили прирост производительности, что показывает смысл применения индексов в нашей базе данных